IMPA Tech

Computação Gráfica – 2026.1

Profa. Asla Medeiros de Sá

Vetorização de Curvas de Bézier por Aproximação

Modernização e Otimização do Algoritmo de Schneider (1990)

Pedro Rodrigues Alberti
Pedro Miguel Rocha Santos

Rio de Janeiro, Maio de 2026

1 Introdução

Este relatório descreve o desenvolvimento teórico e a implementação do trabalho prático da disciplina Computação Gráfica (2026.1), enquadrado no Tema 4: Recuperação de Modelos a partir de Dados. O objetivo central é a vetorização de contornos e trajetórias discretas por meio do ajuste de curvas de Bézier cúbicas, seguindo o algoritmo proposto por Philip J. Schneider (1990).

O trabalho tem como ponto de partida uma implementação de referência em Python 2 disponível no repositório fitCurves (Poplawski). Este projeto moderniza essa base: o código foi reestruturado para Python 3, as operações matemáticas centrais foram vetorizadas com NumPy e a interface gráfica interativa foi melhorada para facilitar a exploração do algoritmo.

A atualização do código foi fundamental para consolidarmos o entendimento do algoritmo e sua implementação prática. Ao modernizar a base original, evidenciamos a evolução das ferramentas numéricas nos últimos 15 anos, substituindo lógicas legadas por operações vetorizadas e padrões contemporâneos. Acreditamos que esse processo conecta a teoria de 1990 à eficiência da engenharia de software científica atual.

2 Fundamentação Teórica

2.1 Curvas de Bézier Cúbicas

Uma curva de Bézier de grau n é definida como combinação convexa de $n+1$ pontos de controle $V_0, V_1, \ldots, V_n$ por meio dos polinômios de Bernstein:

$$Q(t) = \sum_{i=0}^{n} V_i \, B_i^n(t), \quad t \in [0,1],$$

onde os polinômios de Bernstein de grau n são dados por

$$B_i^n(t) = \binom{n}{i} t^i (1-t)^{n-i}, \quad i = 0, \ldots, n.$$

No caso cúbico ($n = 3$), a curva possui quatro pontos de controle V_0, V_1, V_2, V_3, e admite avaliação eficiente pelo algoritmo recursivo de Casteljau, baseado na propriedade de recursão dos polinômios de Bernstein:

$$B_i^n(t) = (1-t)\, B_i^{n-1}(t) + t\, B_{i-1}^{n-1}(t).$$

A derivada de ordem k de uma curva de Bézier é ela própria uma curva de Bézier nos operadores de diferença avançada dos pontos de controle:

$$\frac{d^k}{dt^k} Q(t) = \frac{n!}{(n-k)!} \sum_{i=0}^{n-k} \Delta^k V_i \, B_i^{n-k}(t),$$

com $\Delta^1 V_i = V_{i+1} - V_i$ e $\Delta^k V_i = \Delta^{k-1} V_{i+1} - \Delta^{k-1} V_i$. Em particular, para $t = 0$ tem-se $Q'(0) = n(V_1 - V_0)$, de modo que a direção tangente no extremo esquerdo é determinada inteiramente pelo segmento $V_0 V_1$, e analogamente no extremo direito.

2.2 Parametrização por Comprimento de Corda

Dado um conjunto de pontos digitalizados $\{d_i\}_{i=1}^n$, é necessário associar a cada ponto um valor paramétrico $u_i \in [0,1]$. A parametrização por comprimento de corda define

$$u_1 = 0, \qquad u_i = u_{i-1} + \|d_i - d_{i-1}\|, \quad i = 2, \ldots, n,$$

seguido de normalização $u_i \leftarrow u_i/u_n$. Esta escolha é motivada pela hipótese de que o parâmetro t cresce aproximadamente proporcionalmente ao comprimento de arco, o que é razoável para curvas suaves sem inflexões abruptas. A parametrização por comprimento de corda serve como estimativa inicial, suficientemente boa para iniciar o processo de ajuste, mas em geral não ótima.

2.3 Ajuste por Mínimos Quadrados

Fixados os valores paramétricos $\{u_i\}$ e as tangentes unitárias $\hat{t}_1$ e $\hat{t}_2$ nos extremos, os quatro pontos de controle satisfazem:

- $V_0 = d_1$ e $V_3 = d_n$ (pontos de controle fixos nos extremos);
- $V_1 = \alpha_1 \hat{t}_1 + V_0$ e $V_2 = \alpha_2 \hat{t}_2 + V_3$ (pontos interiores ao longo das tangentes).

O problema se reduz a encontrar $\alpha_1, \alpha_2 \in \mathbb{R}$ que minimizem o erro quadrático total:

$$S(\alpha_1, \alpha_2) = \sum_{i=1}^{n} \big\| d_i - Q(u_i) \big\|^2.$$

Como $Q(u_i)$ é linear em α_1 e α_2, a condição de optimalidade $\partial S/\partial \alpha_j = 0$ produz um sistema linear 2×2:

$$C \begin{pmatrix} \alpha_1 \\ \alpha_2 \end{pmatrix} = X,$$

onde, definindo $A_{i,j} = \hat{t}_j \, B_j^3(u_i)$,

$$C = \begin{pmatrix} \sum_i A_{i,1}^2 & \sum_i A_{i,1} A_{i,2} \\ \sum_i A_{i,1} A_{i,2} & \sum_i A_{i,2}^2 \end{pmatrix}, \qquad X_j = \sum_{i=1}^{n} \Big(d_i - V_0 B_0^3(u_i) - V_3 \big(B_3^3(u_i) \big) \Big) \cdot A_{i,j}.$$

A solução analítica é obtida pela Regra de Cramer:

$$\alpha_1 = \frac{\det(X \; C_2)}{\det(C_1 \; C_2)}, \qquad \alpha_2 = \frac{\det(C_1 \; X)}{\det(C_1 \; C_2)}.$$

Caso o determinante de C se anule (situação degenerada como em apenas dois pontos), adota-se a heurística de posicionar os pontos interiores a $\|d_n - d_1\|/3$ dos extremos, ao longo das respectivas tangentes.

2.4 Reparametrização por Newton–Raphson

A parametrização por comprimento de corda é uma aproximação grosseira do parâmetro verdadeiro. Uma vez ajustada a curva $Q(t)$ com os parâmetros correntes $\{u_i\}$, pode-se refinar cada u_i resolvendo o problema de projeção:

$$\big[Q(t) - d_i\big] \cdot Q'(t) = 0,$$

que caracteriza a projeção ortogonal de d_i sobre a curva. Aplicando um passo de Newton–Raphson a esta equação escalar em t:

$$u_i \leftarrow u_i - \frac{\big[Q(u_i) - d_i\big] \cdot Q'(u_i)}{\big[Q'(u_i)\big]^2 + \big[Q(u_i) - d_i\big] \cdot Q''(u_i)}.$$

A reparametrização é computacionalmente barata e converge rapidamente. O processo de reparametrização e reajuste é repetido até que o erro máximo caia abaixo da tolerância ε definida pelo usuário ou que se esgote o número máximo de iterações.

2.5 Estimação das Tangentes nos Extremos

As tangentes unitárias $\hat{t}_1$ e $\hat{t}_2$ nos extremos da subcurva a ser ajustada podem ser estimadas por diferentes estratégias, dependendo da etapa do algoritmo e das características dos dados:

(i) média ponderada dos vetores unitários formados pelos primeiros (ou últimos) k pontos vizinhos ao extremo, com pesos $w_j = 1/j$ decrescentes com a distância ao extremo;

(ii) regressão linear local por mínimos quadrados na vizinhança do extremo, mais robusta para dados ruidosos;

(iii) na subdivisão recursiva, a tangente no ponto de corte d_s é estimada pelos vizinhos imediatos:

$$\hat{t}_s = \frac{d_{s-1} - d_{s+1}}{\|d_{s-1} - d_{s+1}\|}$$

garantindo continuidade G^1 na junção entre segmentos.

2.6 Subdivisão Recursiva

O algoritmo global opera de forma recursiva. Dado um conjunto de pontos $\{d_i\}$ e uma tolerância ε:

1. Estime as tangentes nos extremos e atribua parâmetros por comprimento de corda.
2. Resolva o sistema de mínimos quadrados para obter α_1, α_2 e construa $Q(t)$.
3. Compute o erro máximo $e^* = \max_i \|d_i - Q(u_i)\|$ e identifique o índice i^* do ponto mais distante.
4. Se $e^* < \varepsilon$: aceite o segmento e encerre.

5. Se $e^* < \psi$ (limiar intermediário, tipicamente $\psi = 4\varepsilon$): tente reparametrizar por Newton–Raphson e volte ao passo 2; caso o número máximo de iterações seja atingido sem convergência, vá ao passo 6.

6. Subdivida os pontos em $d_1, \ldots, d_{i^*}$ e $d_{i^*}, \ldots, d_n$; estime a tangente em d_{i^*} e aplique o algoritmo recursivamente a cada subconjunto.

O resultado é uma representação adaptativa: trechos suaves da curva são cobertos por segmentos longos, enquanto regiões com mudanças abruptas de direção geram subdivisões mais densas.

3 Implementação

A principal contribuição arquitetural foi a substituição dos laços procedurais da implementação original por operações de *broadcast* e `einsum` do NumPy, em particular na montagem das matrizes C e X em `_generate_bezier`. Isso reduz a complexidade do código e melhora a eficiência para conjuntos grandes de pontos.

Ao longo da implementação, foram feitas mudanças com os seguintes propósitos:

- **Robustez:** Adição de verificações de erro e garantias de formatação adequada dos dados de entrada com `np.asarray(..., dtype=float)`.
- **Legibilidade:** Incorporação de técnicas de otimização de sintaxe e funções auxiliares, além de uma modularização que torna o código autodocumentado.
- **Eficiência:** O uso de operações vetorizadas do NumPy resulta em código otimizado em termos de memória e processamento.
- **Estabilidade Numérica:** Tratamento de casos degenerados (denominadores nulos) e adoção de uma heurística de *epsilon* dinâmico para operações de ponto flutuante.

O projeto é composto por dois módulos principais: `bezier.py` e `fitCurves.py`.

3.1 Módulo `bezier.py` — Avaliação da Curva

As três funções do módulo avaliam a geometria da curva de Bézier cúbica. Todas aceitam o parâmetro t como escalar ou *array*, permitindo avaliação vetorizada:

`q`: Avalia a posição na curva.

`qprime`: Avalia a primeira derivada (tangente).

`qprimeprime`: Avalia a segunda derivada (curvatura).

Seguem imagens dos códigos utilizados como referência e da implementação desenvolvida, evidenciando o enfoque dado à legibilidade, organização e controle da formatação dos dados.

```python
def q(ctrl_poly: np.ndarray, t: float | np.ndarray) -> np.ndarray:
    """Avalia a curva de Bézier cúbica no(s) parâmetro(s) t.

    Args:
        ctrl_poly: Array (4, 2) com os pontos de controle [P0, P1, P2, P3].
        t:         Escalar ou array de parâmetros em [0, 1].

    Returns:
        Ponto(s) na curva como array NumPy.
    """
    ctrl_poly = np.asarray(ctrl_poly, dtype=float)
    t = np.asarray(t, dtype=float)
    t_ = 1.0 - t
    return (
        t_**3              * ctrl_poly[0]
        + 3 * t_**2 * t    * ctrl_poly[1]
        + 3 * t_    * t**2 * ctrl_poly[2]
        +             t**3 * ctrl_poly[3]
    )
```

(a) Nossa implementação

```python
# evaluates cubic bezier at t, return point
def q(ctrlPoly, t):
    return (1.0-t)**3 * ctrlPoly[0] + 3*(1.0-t)**2 * t * ctrlPoly[1] + 3*(1.0-t)* t**2 * ctrlPoly[2] + t**3 * ctrlPoly[3]


# evaluates cubic bezier first derivative at t, return point
def qprime(ctrlPoly, t):
    return 3*(1.0-t)**2 * (ctrlPoly[1]-ctrlPoly[0]) + 6*(1.0-t) * t * (ctrlPoly[2]-ctrlPoly[1]) + 3*t**2 * (ctrlPoly[3]-ctrlPoly[2])


# evaluates cubic bezier second derivative at t, return point
def qprimeprime(ctrlPoly, t):
    return 6*(1.0-t) * (ctrlPoly[2]-2*ctrlPoly[1]+ctrlPoly[0]) + 6*(t) * (ctrlPoly[3]-2*ctrlPoly[2]+ctrlPoly[1])
```

(b) Implementação original

Figura 1: Comparação entre a nossa implementação e a original nas funções de avaliação

3.2 Módulo `fitCurves.py` — Ajuste Automático

Este módulo implementa o algoritmo de ajuste através de funções encadeadas:

`fit_curve`: Valida a entrada, calcula as tangentes nas extremidades e delega ao núcleo recursivo.

`_fit_cubic`: Tenta ajustar uma única curva ao segmento de pontos. Se o erro for inaceitável, divide o segmento no ponto de maior desvio e chama a si mesmo recursivamente.

`_generate_bezier`: Calcula os pontos de controle internos P_1 e P_2 via mínimos quadrados. Esta é a função mais modificada: Os laços em Python foram substituídos por operações com `einsum`, executadas internamente em código otimizado em C. A seguir, uma visualização comparando as duas abordagens.

```python
def _generate_bezier(points: np.ndarray, parameters: list[float], left_tangent: np.ndarray, right_tangent: np.ndarray) -> BezierCurve:
    """Calcula os pontos de controle internos via mínimos quadrados (método de Schneider)."""

    u = np.asarray(parameters) # (N,)
    t_ = 1.0 - u # (N,)

    # Matriz A (N, 2, 2) onde A[i][0] é o vetor associado a left_tangent e A[i][1] ao right_tangent para o parâmetro u[i]
    # Usada por Schneider para auxiliar os cálculos
    A = np.zeros((len(u), 2, 2))
    A[:, 0] = (left_tangent  * (3 * t_**2 * u)    [:, np.newaxis])
    A[:, 1] = (right_tangent * (3 * t_    * u**2)  [:, np.newaxis])

    # Montagem da matriz C usando cálculo otimizado em tesnores do numpy (evitando loops explícitos como o da implementação original)
    C = np.einsum("ijk,ijl->kl", A, A)          # C[k,l] = sum_i dot(A[i,k], A[i,l])

    # Termo independente: ponto observado menos Bézier degenerada (P0..P0..P3..P3)
    bez_degenerate = np.array([points[0], points[0], points[-1], points[-1]])
    tmp = points - np.array([q(bez_degenerate, ui) for ui in u])  # (N, 2)

    # Montagem da matriz X usando cálculo otimizado em tesnores do numpy
    X = np.einsum("ijk,ij->k", A, tmp)          # X[k] = sum_i dot(A[i,k], tmp[i])
```

(a) Nossa implementação

```python
def generateBezier(points, parameters, leftTangent, rightTangent):
    bezCurve = [points[0], None, None, points[-1]]

    # compute the A's
    A = zeros((len(parameters), 2, 2))
    for i, u in enumerate(parameters):
        A[i][0] = leftTangent  * 3*(1-u)**2 * u
        A[i][1] = rightTangent * 3*(1-u)    * u**2

    # Create the C and X matrices
    C = zeros((2, 2))
    X = zeros(2)

    for i, (point, u) in enumerate(zip(points, parameters)):
        C[0][0] += dot(A[i][0], A[i][0])
        C[0][1] += dot(A[i][0], A[i][1])
        C[1][0] += dot(A[i][0], A[i][1])
        C[1][1] += dot(A[i][1], A[i][1])

        tmp = point - bezier.q([points[0], points[0], points[-1], points[-1]], u)

        X[0] += dot(A[i][0], tmp)
        X[1] += dot(A[i][1], tmp)
```

(b) Implementação original

Figura 2: Comparação entre a nossa implementação e a original na geração da Bezier

`_newton_raphson_root_find`: Refina iterativamente o parâmetro u de cada ponto minimizando sua distância à curva. A atualização em cada iteração é dada por:

$$u \leftarrow u - \frac{\big[Q(u) - d_i\big] \cdot Q'(u)}{|Q'(u)|^2 + \big[Q(u) - d_i\big] \cdot Q''(u)}$$

onde Q, Q' e Q'' são avaliados pelas funções `q`, `qprime` e `qprimeprime` do módulo `bezier.py`.

A interface gráfica em `demo.py` é uma adaptação modernizada da demo original de

Poplawski, portada de `Tkinter` (Python 2) para `tkinter` (Python 3) e estendida com novas funcionalidades. O comportamento original — adicionar pontos com o botão esquerdo, mover pontos diretamente pelo canvas e removê-los com o botão do meio — foi preservado. Além disso, foram adicionados suporte à remoção pelo botão direito do mouse (mantendo o botão do meio por compatibilidade), um botão para limpar completamente o canvas e uma barra de status que exibe, em tempo real, o número de pontos ativos, de segmentos gerados e o valor corrente de `max_error`. A tolerância do ajuste também pode ser modificada dinamicamente por meio do *spinbox* associado ao campo `Max Error`.

Por fim, nossa implementação se distanciou da versão original principalmente na função `generateBezier`, na qual os laços explícitos foram substituídos por operações vetorizadas com `broadcasting` e `einsum`. Além disso, realizamos outras modificações pontuais com o objetivo de tornar o código mais legível, modular e robusto.

4 Resultados

A implementação foi validada em três conjuntos de pontos com características distintas.

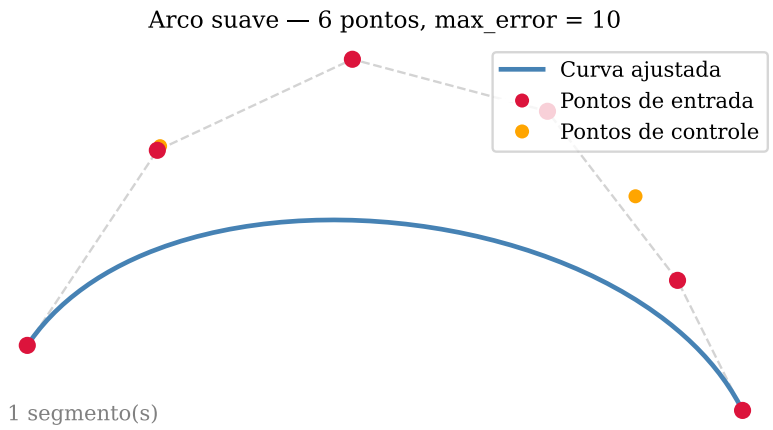

Figura 3: Arco suave com 6 pontos. O algoritmo converge em um único segmento.

A Figura 4 ilustra o efeito da tolerância sobre o número de segmentos gerados para o mesmo conjunto de pontos irregulares. Com `max_error = 2`, o algoritmo gera 5 segmentos para respeitar a tolerância imposta; ao aumentar para 15 ou 60, um único segmento passa a ser suficiente.

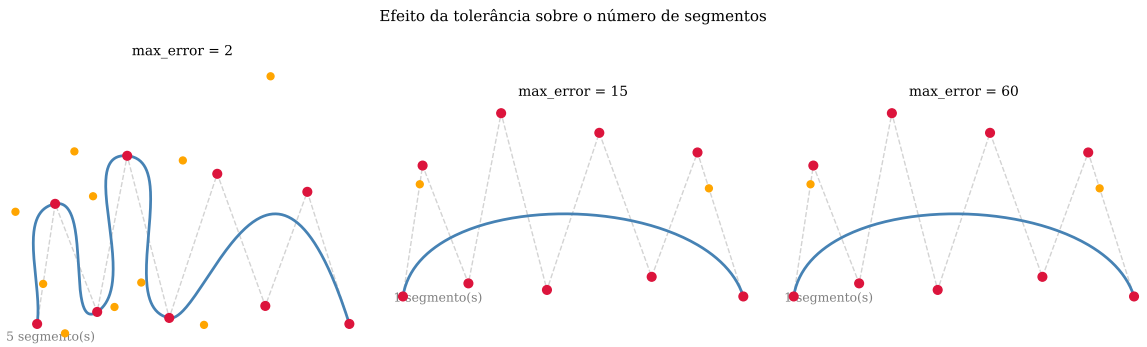

Figura 4: Efeito de `max_error` sobre o ajuste. A redução da tolerância força o algoritmo a subdividir o conjunto de pontos em mais segmentos.

Para dados com variações abruptas de direção e tolerância restrita, o algoritmo aplica subdivisão recursiva intensiva. Com `max_error = 2`, o ajuste ao contorno em ziguezague resultou em 10 segmentos, cada um cobrindo uma oscilação local.

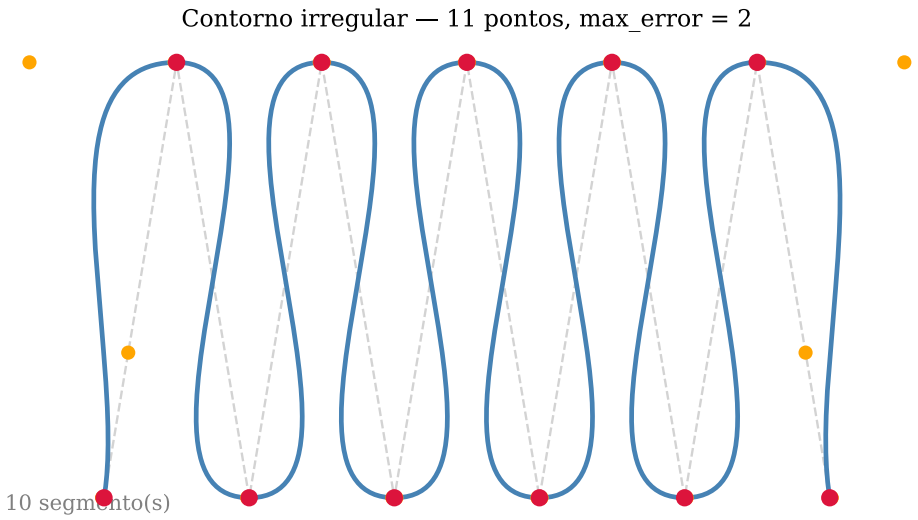

Figura 5: Contorno irregular com 11 pontos e `max_error = 2`. A subdivisão recursiva gera 10 segmentos para cobrir as oscilações abruptas dentro da tolerância.

5 Conclusão

A reimplementação do algoritmo de Schneider (1990) revelou-se uma experiência bastante enriquecedora. O processo de adaptar um código legado aos padrões atuais exigiu compreensão detalhada de cada etapa do algoritmo, consolidando na prática conceitos de curvas paramétricas, mínimos quadrados e otimização, fundamentais em computação gráfica e em diversos métodos numéricos. Além disso, o projeto proporcionou um entendimento mais profundo das curvas de Bézier, tópico importante da disciplina.

Também se tornou evidente a evolução do ecossistema numérico do Python desde 2012: operações que anteriormente demandavam laços explícitos e implementações fortemente procedurais podem hoje ser expressas de maneira mais compacta, legível e eficiente com o uso de NumPy.

Referências

[1] Volker Poplawski. FitCurves: Python implementation of Philip J. Schneider's "Algorithm for Automatically Fitting Digitized Curves" from the book "Graphics Gems". `https://github.com/volkerp/fitCurves`. Acesso em: 14 maio 2026.

[2] Philip J. Schneider. An Algorithm for Automatically Fitting Digitized Curves. In Andrew Glassner, editor, *Graphics Gems*. Academic Press, 1990.